API Security Risk Analysis Report

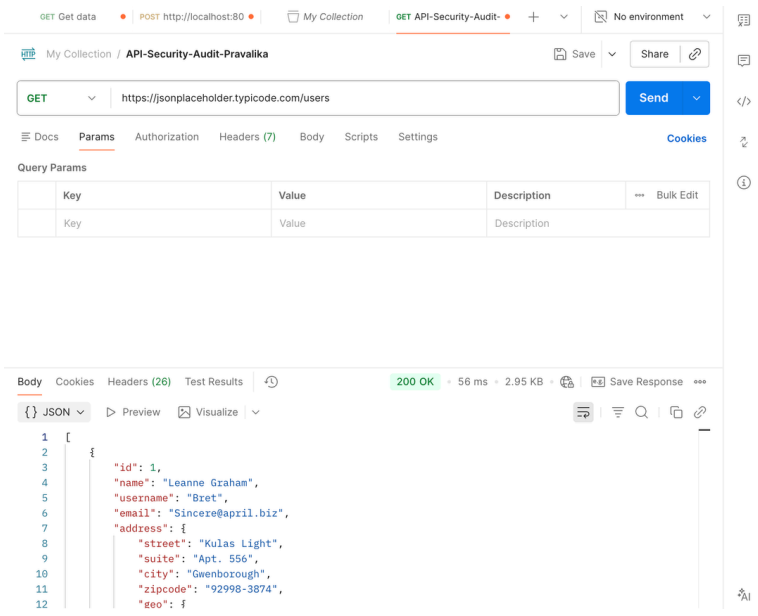
Target Scope: API Security Audit for JSONPlaceholder (Test API)

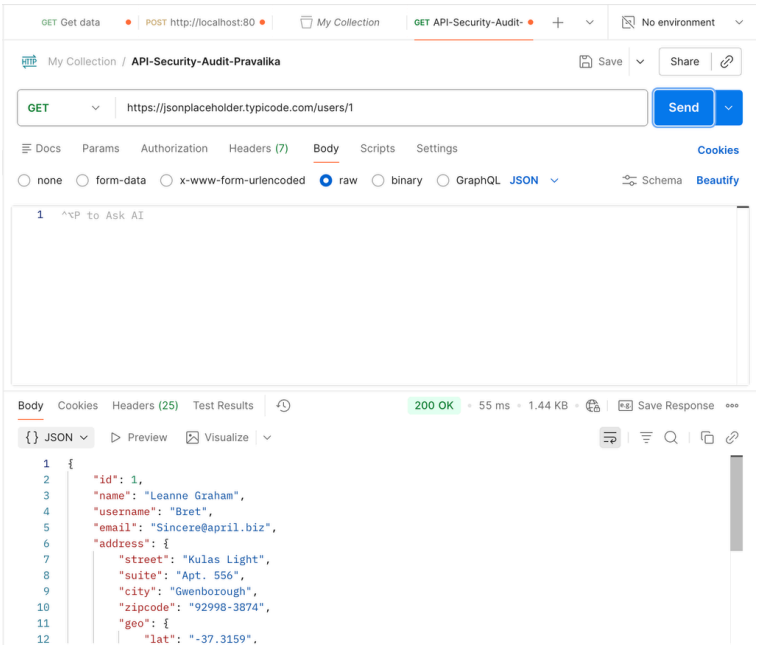
Evidence Reference: In the given below images

Prepared By: Devarshetty Pravalika

Date: June 8, 2026

Overview: This assessment performs a read-only security analysis of the JSONPlaceholder API. By inspecting headers, response payloads, and input handling, this report identifies systemic security misconfigurations and data exposure risks to provide actionable remediation steps for developers.





Executive Summary & Methodology

Executive Summary:

This audit identifies key security gaps within the target API. The findings highlight potential risks regarding information disclosure, lack of input validation, and missing security headers, which could impact data integrity and application security in a production environment.

Methodology:

- **Tools Used:** Postman and Browser DevTools.
- **Target Base URL:** [https://jsonplaceholder.typicode.com] (https://jsonplaceholder.typicode.com)
- **Scope:** Analyzed endpoints including GET /users, GET /users/1, and POST /users.
- **Approach:** Conducted non-invasive, read-only analysis focusing on request headers, response metadata, and server-side input handling.

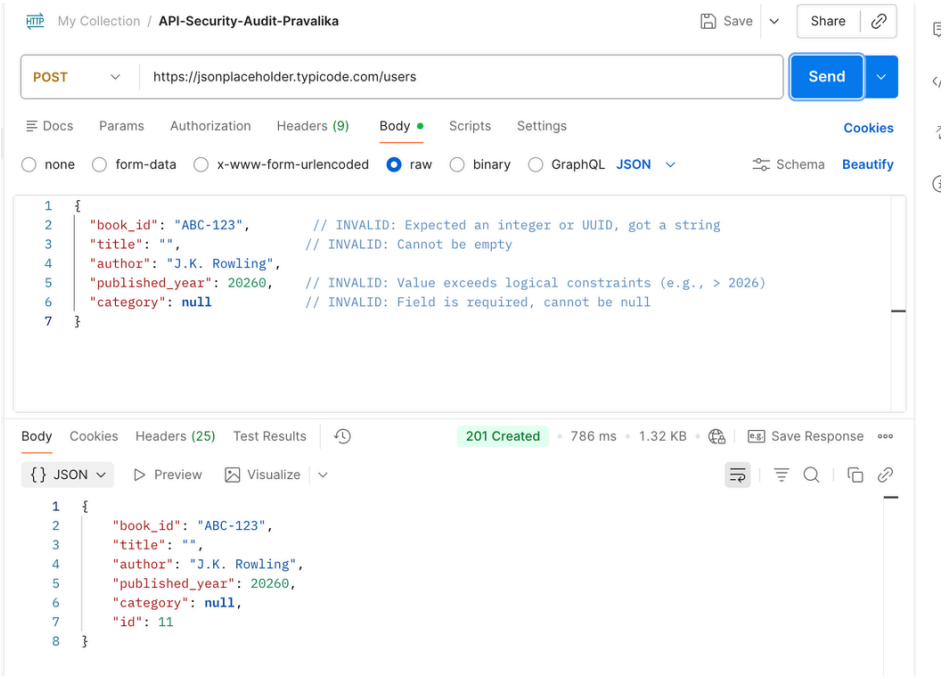
Detailed Findings Table

Risk Finding	Severity	Business Impact	Remediation Suggestion
Information Disclosure	Low	Exposes internal technology stack (X-Powered-By: Express).	Disable/remove technical headers.
Lack of Input Validation	Medium	Allows malformed data injection.	Implement strict DTO/schema validation.
Missing Security Headers	Medium	Susceptibility to Clickjacking/XSS.	Implement CSP and HSTS headers.
Excessive Data Exposure	Medium	Exposes sensitive user PII by default.	Apply data filtering/field-level masking.

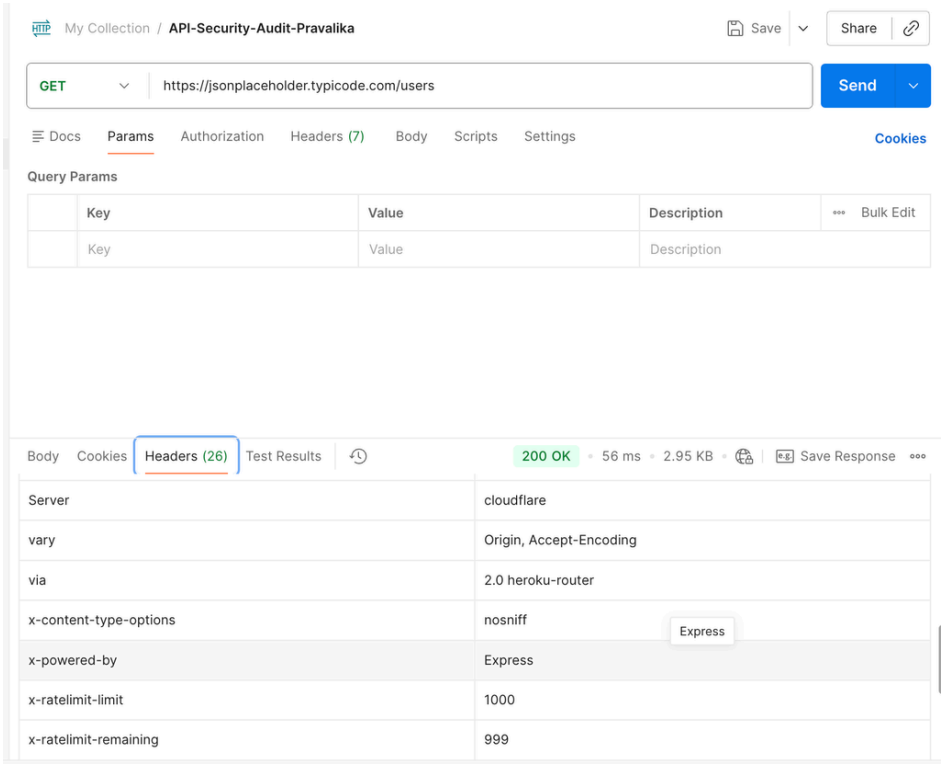
Technical Breakdown (Evidence-Based)

1.Lack of Input Validation:

As evidenced in below given screenshot image, the API returned a 201 Created status for an intentionally invalid body, demonstrating a failure to validate data integrity.

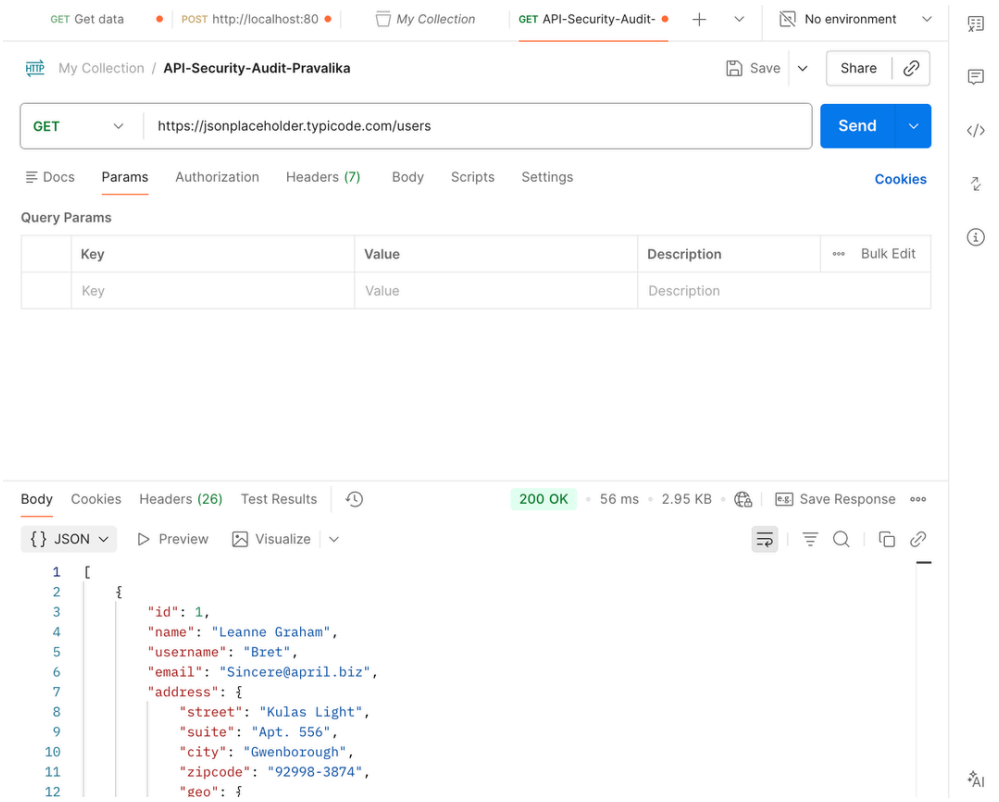


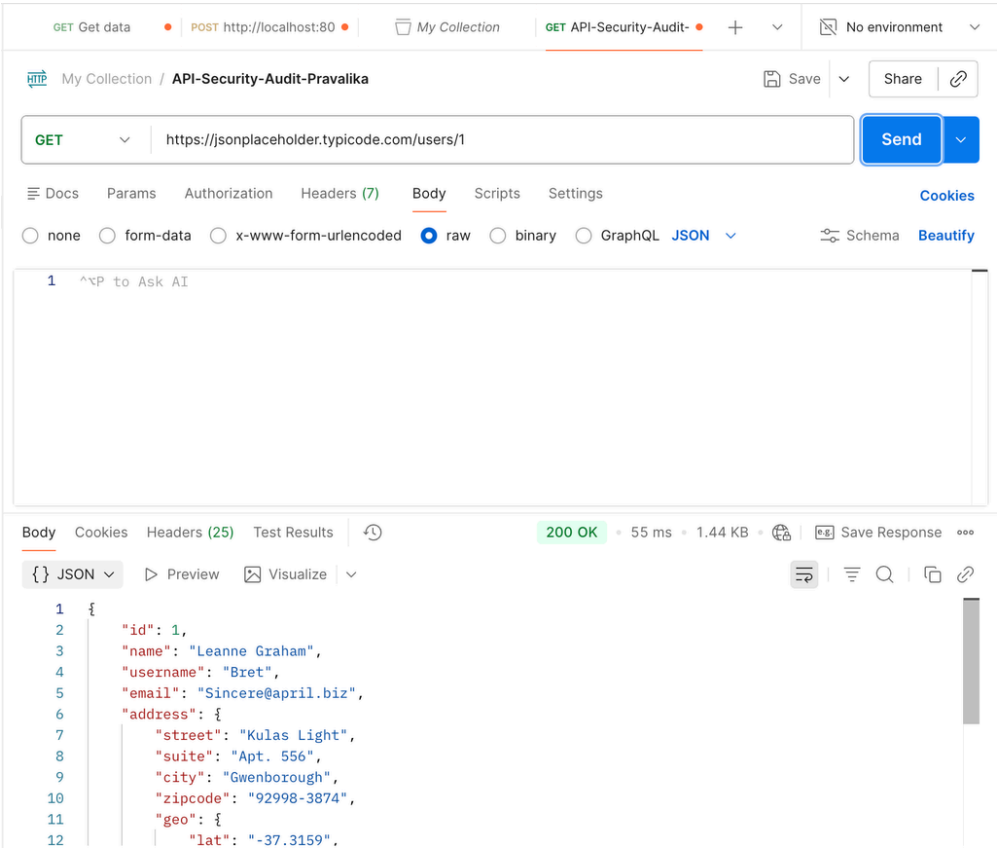
2.Security Misconfiguration: As evidenced in below given screenshot image confirms the disclosure of X-Powered-By: Express, which aids attackers in footprinting the target stack.



3.Data Exposure:

Below given Screenshot images show that full user details, including geolocation and personal addresses, are returned without restrictive access controls.





Conclusion & Recommendations

While the API effectively demonstrates core functionality for testing environments, it contains significant security misconfigurations that would pose substantial risks if deployed in a production SaaS ecosystem. To evolve this into a robust, enterprise-grade architecture, the following hardening measures are recommended:

- **Implement Comprehensive Server-Side Validation:** Move beyond basic checks by enforcing strict schema validation on all incoming request bodies and query parameters. This prevents injection attacks and ensures that only sanitized, expected data types reach the application layer.
- **Enforce Hardened Security Headers:** Integrate security-focused HTTP response headers—such as Content Security Policy (CSP), HSTS, and X-Content-Type-Options—to mitigate cross-site scripting (XSS) and man-in-the-middle (MITM) risks.
- **Adopt Rigorous Data Filtering & Sanitization:** Apply a "deny-all" approach to data ingestion. By implementing centralized middleware for sanitization, you ensure consistent filtering of malicious inputs before they reach business logic or persistent storage.
- **Strengthen Authentication and Authorization:** Given the requirements for high-density, secure applications, transition to industry-standard authentication flows (such as those used with NextAuth.js) to enforce granular role-based access control (RBAC), ensuring that users can only interact with authorized data segments.
- **Integrate Threat Monitoring:** Leverage proactive security operations methodologies. Aligning your infrastructure logs with frameworks like MITRE ATT&CK or utilizing Sigma rules for detection will provide the visibility necessary to identify and respond to potential threats in real time.